

IDC TO B White Paper

Public nodes & owned nodes · Token model & “mining” · Risk control & audit loop

Version: v1.0 | Date: 2026-05-08

Note: First edition (**v1.0.1** alignment: terminology, illustrative tables, and mapping framework added; all numeric values remain subject to deployment configs and runtime policy.)

Table of contents

1. [Part I — General](#) (Abstract, Background, Terms, Overview, Economy)
 2. [Part II — Token model & “mining”](#)
 3. [Part III — Nodes, risk control & appendix](#)
- [1 Abstract](#)
 - [2 Background & objectives](#)
 - [3 Terms & definitions](#)
 - [4 System overview](#)
 - [5 Economic model & settlement rules](#)
 - [II-1 Preface](#) · [II-2 Design](#) · [II-3 Supply \(growth\)](#) · [II-4 Supply \(maturity\)](#) · [II-5 Allocation](#) · [II-6 Mining \(growth\)](#) · [II-7 Mining \(maturity\)](#) · [II-8 Mapping](#) · [II-9 Dynamic B](#) · [II-10 Nodes](#) · [II-11 Utility](#) · [II-12 Identity](#) · [II-13 Roadmap](#)
 - [6 Public nodes](#) · [7 IDC nodes](#) · [8 Referral](#) · [9 Risk & audit](#) · [10 Roadmap](#) · [11 FAQ](#)

1 Abstract

IDC TO B is a token-style incentive system oriented to growth and sustainable yield. It maps *verifiable behaviour* to stable, explainable, auditable payout triggers, and uses a **two-layer capacity model** (public nodes + owned IDC nodes) so new users can earn IDC without running infrastructure, while advanced users can accelerate through node-backed rates.

The design addresses: (1) settlement uncertainty under scripted traffic; (2) invite-only growth without durable capacity or clear rules; (3) operational friction in node deployment, observability, and trust. The stack combines verification, check-in logic, node rate bonuses, tiered

referral rewards, and risk/audit controls—plus self-checks, scheduled jobs, and signed Agent reports for observability.

Principle: every credit must trace to *behaviour evidence + validation outcome + settlement definition*, reproducible from audit logs when needed.

2 Background & objectives

2.1 Two tensions

A — Growth vs. settlement certainty. Pure invite spikes can hollow teams and inflate metrics. Check-in and validation anchor yield to explainable facts.

B — Low-friction onboarding vs. quality capacity. Requiring self-hosted nodes for everyone raises the bar; relying only on shared pools is hard to govern. Public nodes provide a *base slot*; IDC nodes add explainable uplift.

2.2 Objectives

1. Let users reach first IDC quickly after basic onboarding.
2. Make node uplift observable and explainable.
3. Make referral economics rule-based and evidence-backed.
4. Treat risk control and audit as product defaults, not post-launch patches.

3 Terms & definitions

Terms below are scoped to this white paper; if UI copy differs, follow in-product configuration text and audit definitions.

Term	Definition
IDC	In-platform unit for incentive settlement (check-in, referral, node contribution, etc.). Not described here as a standalone public-chain native asset or fundraising instrument.
B	Global base-rate factor (or budget-pool equivalent), adjustable by phase, milestone, and configuration.

S	Security/trust constraints or multipliers—e.g. captcha passed, valid check-in, risk score in normal range.
Agent	Software on an owned IDC node performing enroll/report, metrics collection, and signed requests.
Public node	Platform-provided shared base capacity (when policy allows), lowering time-to-first-yield.
Valid settlement event	A business fact that passes rules and risk controls and can trigger accrual or referral (e.g. valid check-in settlement, successful signed node report)—distinct from “registration only”.
configs	Deployment configuration store (rates, caps, feature flags). Ratios in this paper are narrative frames; live values follow configs and change records.

Note: L1/L2/L3 denote invite tiers; M, I, E, N, A, X are symbols used in Part II formulas.

4 System overview

4.1 User journey

1. Register / sign in with verification and anti-bot controls.
2. Complete check-in to accrue IDC (optionally via public-node base capacity when enabled).
3. Bind IDC nodes and deploy Agents; reports unlock node uplift.
4. Grow via invite links (three-tier relationships) with rule-based referral accounting.
5. Continuous accrual may be driven by scheduled jobs even when the user is offline.

4.2 Technical loop

Split **web behaviour** (check-in, invites) from **background work** (health checks, discovery, accrual). Admin “self-check” surfaces task scripts and status. Nodes use enroll/report with signed payloads; offline nodes degrade with alerts.

Observability triad: state (online, enrolled, last report & errors); rate (base, node bonus, team referral splits); evidence (check-in logs, audit trails, signatures).

5 Economic model & settlement rules

5.1 Define the rules first

Define *trigger*, *formula*, and *exceptions* (failure, delay, rollback, risk blocks) before scaling traffic. Rule changes ship with version numbers and effective times surfaced to users.

5.2 Balance + rate + windows

Express IDC as **balance** (“how much I have”) and **rate** (“how fast it grows”), decomposed in the user centre. Use a single server timezone and explicit “settlement day” in audit logs to avoid duplicate or missed daily boundaries.

5.3 Node bonuses & caps

Combine *tiers*, *per-node/per-user caps*, and *stability coefficients* (uptime streaks, offline counts). Persist parameter snapshots (version, thresholds, caps) for replay and disputes.

5.4 Referral economics

Tie rewards to **valid settlement events** (downline check-in settled, node report accepted, gated tasks passed)—not raw registration counts. Show daily vs cumulative contribution with source splits (check-in vs node).

Part II — Token model & “mining” mechanism

Important: IDC is an in-platform accounting unit for incentives, not a claim about an independent public-chain native asset. “Mainnet / pre-mainnet” wording here is *phase analogy* only (growth vs maturity) and implies no affiliation with third-party projects. Fundraising, secondary pricing, or public solicitation in the name of IDC requires lawful qualification and official announcements; such activities are outside the scope of this document.

II-1 Preface: why a dedicated chapter

Public “supply & mining” chapters answer: how scarcity coexists with growth; how incentives align with diverse contributions; how issuance avoids runaway inflation or rigidity as scale shifts. IDC TO B needs the same clarity—supply philosophy, allocation, abstract formulas, dynamic factors, and risk boundaries.

Structure follows common white-paper patterns; **all numbers, caps, ratios, and lock-up/inflation flags are governed by configs and runtime policy**. Illustrations here are explanatory, not a guarantee for any single deployment.

II-2 Token design & network goals

IDC anchors utility to: verifiable behaviour; network/security (captcha, rate limits, audits); capacity via Agents and health tiers; collaboration via L1/L2/L3 referral. This part covers issuance rhythm, how users earn IDC (we keep the colloquial “mining” for incentive accrual), and design intent (bootstrap the network, reward utility, limit abuse).

II-3 IDC supply — growth phase

Priorities: user scale, behaviour distribution, risk readiness, broad IDC reach. Outwardly: public nodes + base check-in lower onboarding friction; inwardly: captcha providers (e.g. Geetest / reCAPTCHA v3) reduce scripted abuse.

Milestone-style global rate adjustments (e.g. when active users cross thresholds) avoid a single fixed rate that either dilutes incentives or overheats inflation expectations. Elastic pools may omit a hard cap early, but require traceable audits, circuit breakers, and a path to the next phase.

II-4 IDC supply — maturity & constraints

At scale, goals expand to diversified contribution and long-term utility. Supply side typically needs: a clear max supply *or* annual budget (or both); decaying or dynamic allocation curves; distinct treatment of nodes, invites, and future modules.

Transition from “scale-only behaviour issuance” to **budget pool + dynamic B + weighting**: allocate daily/weekly budgets under a total cap.

Optional modules (lock-up boosts, liquidity reserves, ecosystem funds) are product options—disclose under compliance and log auditable changes.

II-5 Allocation principles & community quota (illustrative)

The table below is a governance/product illustration (“pie narrative”). Replace with approved numbers in configs.

Pool / direction	Typical use	Notes
Base check-in & public nodes	Onboarding & first-yield path	Bound by settlement day, risk controls, and feature flags—not unlimited “idle farming”.
IDC node bonus	Owned-node contribution	Tied to Agent report, health, resource tier, and configured caps.
Referral (L1/L2/L3)	Team growth incentives	Bound to valid settlement events , not registration-only counts.
Ecosystem / campaigns (optional)	Ops milestones & promos	Budget and enablement live in configs and audit trails.
Risk blocks & rollbacks	Abuse & disputes	Debits/freezes require reproducible logs; major parameter moves carry version + effective time.

If the platform uses a single ledger without on-chain multi-address “pools”, rows above map to *chart-of-accounts* / *operational budgets*, not multisig addresses.

II-6 “Mining” — growth phase (abstract)

Instantaneous incentive rate (IDC per hour or equivalent):

$$M = I(B, S) + E(I)$$

- **M** — total mining rate for the user.

- $I(\mathbf{B}, \mathbf{S})$ — personal base rate from global base $\mathbf{B}$ and trust/safety component $\mathbf{S}$.
- $E(\mathbf{I})$ — team referral term coupled with $\mathbf{I}$.

Interpret $\mathbf{S}$ as account safety and behavioural trust (captcha, valid check-in, normal risk score). Do not copy unrelated variables (e.g. “security circle size”) unless the product implements them.

$\mathbf{B}$ may be piecewise-constant in growth, stepping down at milestones.

II-6.1 Example: $\mathbf{B}$ stepping down with milestones (illustrative)

Phase	Trigger (example, non-binding)	$\mathbf{B}$ adjustment
0	Cold start / seed users	Baseline B_0 from configuration.
1	DAU crosses threshold A_1	Global downshift (e.g. $\times 0.8$) or campaign weight shift.
2	Crosses A_2 (scale or report volume)	Further downshift (e.g. $\times 0.5$) or segmented $\mathbf{B}$.
N	Operator-defined	Real thresholds and multipliers live in configs; table shows rhythm only.

II-7 “Mining” — maturity phase (abstract)

$$\mathbf{M} = I(\mathbf{B}, \mathbf{L}, \mathbf{S}) + E(\mathbf{I}) + \mathbf{N}(\mathbf{I}) + \mathbf{A}(\mathbf{I}) + \mathbf{X}(\mathbf{B})$$

- $\mathbf{L}$ — optional lock-up / commitment multiplier (omit or set 0 if unused).
- $\mathbf{N}(\mathbf{I})$ — IDC node contribution from uptime, reachability, CPU tier, report continuity.
- $\mathbf{A}(\mathbf{I})$ — application utility (compliant effective time / counts; anti-abuse required).
- $\mathbf{X}(\mathbf{B})$ — reserved future contributor classes, expressed relative to $\mathbf{B}$ for unified budgeting.

In maturity, $\mathbf{B}$ behaves like a factor solved from *daily issuance budget / effective total weight* (see II-9).

II-8 Component mapping (public narrative → IDC TO B)

Narrative / component	IDC TO B mapping
Base mining / personal rate	I(B, S) ; public nodes + check-in provide the reachable base path.
“Security circle” / trust (generic)	S : captcha, valid check-in, risk scoring, rate limits.
Team / invite growth	E(I) : L1/L2/L3 tied to valid settlement events.
Node / compute contribution	N(I) : Agent reports, uptime, ports, resource tiers.
Halving / decay narrative	Milestone B cuts, budget caps, phase switches—implemented as config versions and schedulers.
Application utility	A(I) : compliant tasks / effective usage (if enabled).
Ecosystem reserve	X(B) : builders / ops extensions under one budget scale.

II-9 Dynamic global base rate B

With budget Q over a window and total effective weight W , public literature often writes $B \approx \text{daily budget} / (\text{weighted activity} \times \text{time granularity})$. Engineering-wise this becomes: read configs + risk outcomes, write balances and ledgers—requires healthy scheduled jobs (health checks, IDC accrual).

II-10 Node bonus (“node-like contribution”)

Agents report CPU, memory, disk, and online state; the platform maps these to bounded compute credits. Penalize harmful shortcuts (chronic offline, forged reports). Rewards must bind to **signature validation, timestamp windows, and replay protection** (see Part III).

II-11 Application utility

Reward real usage, not idle time alone—log-scaled diminishing returns, cross-app dispersion, anomaly detection (spikes, bot-like timings).

II-12 Identity & risk (KYC analogy)

Today: email + captcha + risk policy. If stronger KYC arrives: publish window & consequences, log rules in configs and audits, avoid discriminatory or unenforceable promises.

II-13 Roadmap (chapter view)

1. Settlement truth: single source for evidence, check-in, risk, referral rules.
2. Node trust: stable reports, observable state, explainable bonuses.
3. Parameter governance: major rate/cap changes via announcements + version records.
4. Utility expansion: more compliant task types and developer tooling.

Appendix: the running system is authoritative; this document is not investment advice.

Part III — Nodes, risk control & appendix

6 Public nodes — “base capacity” for newcomers

6.1 Role

Public nodes are not “no one ever needs their own node”; they let early users feel the yield path and learn settlement rules. Users who need higher yield, control, or stability graduate to IDC nodes.

6.2 Display & privacy

The home page may show online node counts and a redacted address list (e.g. last octets masked). Toggle list visibility in admin; counts can remain for trust.

6.3 Relation to personal yield

When policy allows, post-check-in IDC may flow through public-node base capacity. This should complement—not replace—owned-node economics.

7 IDC node system — how owned nodes raise rate

7.1 Lifecycle

1. **Bind** — user enters IP, port, credentials, display name.
2. **Enroll** — tokenized install command; Agent enrolls the node.
3. **Report** — periodic signed metrics.
4. **Bonus** — platform credits node uplift into the user's rate.
5. **Degrade / recover** — missed reports downgrade state; recovery restores online.

7.2 Trustworthy status

Enrolled ≠ online: **successful report** is the online signal. Show last report time and stale warnings. Credential/port changes should mark the node “pending re-check” to avoid stale green status.

7.3 Explainable UI

Split display: base (public / baseline), node bonus (from online IDC nodes), referral rate—include node count, last report, health, and explicit downgrade reasons (no report, bad signature, clock skew, unreachable).

7.4 Cost vs. reward (decision framing)

Node yield is not a direct reimbursement of server bills; it rewards *sustainable capacity*. Product may show a light “cost awareness” hint. Caps and decay should be configurable; abusive nodes get cooldowns.

7.5 Signed reporting

Use shared-secret HMAC: canonical JSON payload + timestamp; server checks signature, clock skew (e.g. ± 300 s), and optional nonce/fingerprint to limit replay. Define canonical JSON (field order, number formatting, escaping) across languages to avoid “online but always failing signature” incidents.

8 Referral — tiers & cumulative rules

8.1 Tiers

L1 direct, L2 under L1, L3 deeper. Ratios are configured and should be visible in the invite centre.

8.2 Cumulative referral stats

Must be traceable to chain of invites + settlement event + window; explain delays and risk effects; avoid frequent backfills that jitter user-visible numbers. Prefer append-only logs with aggregated reads.

9 Risk control & audit — scalable safety base

9.1 Goals

Stop scripted sign-up, cap verification abuse, throttle abnormal check-ins, detect task anomalies, authenticate node enroll/report.

9.2 Disputes

Provide reproducible evidence: time, user id, parameter digest, validation result, error reason, trace ids when needed—no plaintext secrets in logs; hashes and correlation ids suffice.

9.3 Layered anti-abuse for check-in

Frequency + device fingerprint; behavioural features; high-risk paths trigger extra verification or cooldowns. Roll out with grey modes (log-only → enforce). Return explicit reasons and retry/appeal UX.

9.4 Security baseline

Least privilege for node credentials; never return plaintext secrets to the browser; encrypt sensitive DB fields; keep `node_secret` only on the node host with tight file permissions. CSRF + admin audit for enroll token issuance, node delete, credential changes, referral ratio edits; role-separated admins.

10 Roadmap & iteration strategy

Three threads: **settleable** (evidence capture, exceptions, replay), **scalable** (more task types, finer invite rules), **observable** (health metrics, alerts, ops dashboards). Milestones: stable node reporting; aligned referral ledgers; layered risk rollout; templated growth campaigns.

11 FAQ

Q1: Why “enrolled” but still “pending probe”?

A: Enroll records identity; online depends on successful `report`. Check Agent logs, connectivity, time sync, and signatures.

Q2: Why offline after I changed port/user/password?

A: Credential changes can invalidate probes; the system marks pending re-check until monitoring confirms.

Q3: How do public and owned nodes stack?

A: Typically public supplies base rate/allowance; owned nodes add incremental bonus—see rate breakdown in the user centre.

Q4: Why no referral I can see?

A: Referral needs a valid relationship and a valid settlement event in the window; with no eligible downline activity, referral can be 0.

Q5: Do the tables/ratios in this white paper apply directly in production?

A: No. Tables are illustrative; live values follow configs, change records, and audit logs. Ship parameter changes with version + effective time.

Q6: How do “valid settlement events” relate to referral?

A: Referral should bind to reproducible downstream facts (valid check-in accrual, accepted node reports, etc.), not registration alone. The invite centre should show contribution splits to reduce “I invited but earned nothing” confusion.
